DOCUMENTO DISEÑO CLUB DEPORTIVO SÉNECA GRUPO 7

Miembros:
- Jia Yi Gu - 202520532
- Geronimo Naranjo - 202415840
- Juan Sebastian Chacon - 202513196

1. Contexto y Alcance

1.1 Contexto del proyecto

El Club Deportivo Séneca (CDS) es una institución deportiva que ofrece diferentes disciplinas tanto de conjunto como individuales. Entre sus actividades se encuentran la formación de jugadores, la organización de entrenamientos, los partidos internos y la participación en competencias universitarias. Adicionalmente, el club cuenta con una tienda de artículos deportivos y una cafetería para sus socios. Debido a todas las actividades que son operadas en el club, se necesita un sistema que permita organizar toda la información tanto de jugadores, equipos, competencias, instalaciones y operaciones de venta. Por lo tanto, la idea del proyecto es desarrollar una aplicación que ayude a organizar la administración de datos y permita seguir el desempeño deportivo de los socios así como otros procesos importantes como ventas, empleados y demás.

1.2 Objetivo general

Diseñar e implementar un sistema en Java que permita gestionar las actividades deportivas de los socios, administrativas y comerciales del CDS. De esta manera, se hace uso de la programación orientada a objetos, garantizando la organización y consistencia de la información.

1.3 Alcance del sistema

El sistema del Club Deportivo Séneca busca facilitar la gestión de las actividades deportivas, comerciales y administrativas del club. Permite a los socios inscribirse en equipos y disciplinas individuales, reservar instalaciones, consultar estadísticas y realizar compras. Los empleados pueden gestionar entrenamientos, asistencias, habilitaciones médicas y solicitudes relacionadas con sus turnos. Por su parte, el administrador se encarga de los equipos, competencias, instalaciones, inventario, personal y reportes. También se contempla el almacenamiento de la información para mantener un registro de las actividades del club.

1.4 Restricciones del proyecto

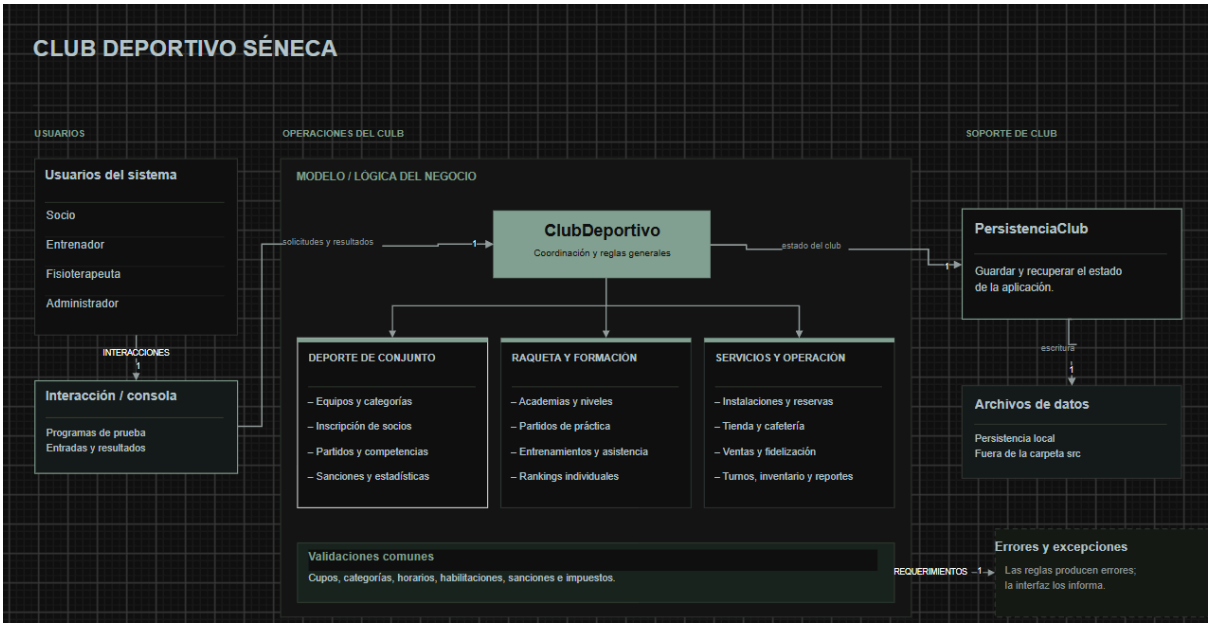
El sistema debe desarrollarse en Java, utilizando programación orientada a objetos y almacenando la información en archivos fuera de la carpeta del código fuente. Todos los usuarios deben contar con un login y contraseña. Además, se deben respetar las reglas del club, como las categorías de edad, los cupos disponibles, las sanciones, la disponibilidad de

instalaciones y los impuestos y descuentos de las compras. También se debe garantizar el personal mínimo requerido durante los turnos.

2. Diseño General del Sistema

2.1 Arquitectura del sistema

Para el diseño del Club Deportivo Séneca se organizó el sistema en diferentes componentes según las funciones que debe cumplir. La clase principal, Club Deportivo, se encarga de coordinar las operaciones del club y relacionar las diferentes áreas del modelo. Esta organización permite separar las responsabilidades y facilita el manejo de la información.



2.2 Organización de los componentes

Interacción: permite ejecutar las operaciones mediante programas de prueba y mostrar sus resultados.

Modelo: contiene la lógica del club, organizada en deportes de conjunto, disciplinas de raqueta, entrenamientos, reservas, tienda y administración.

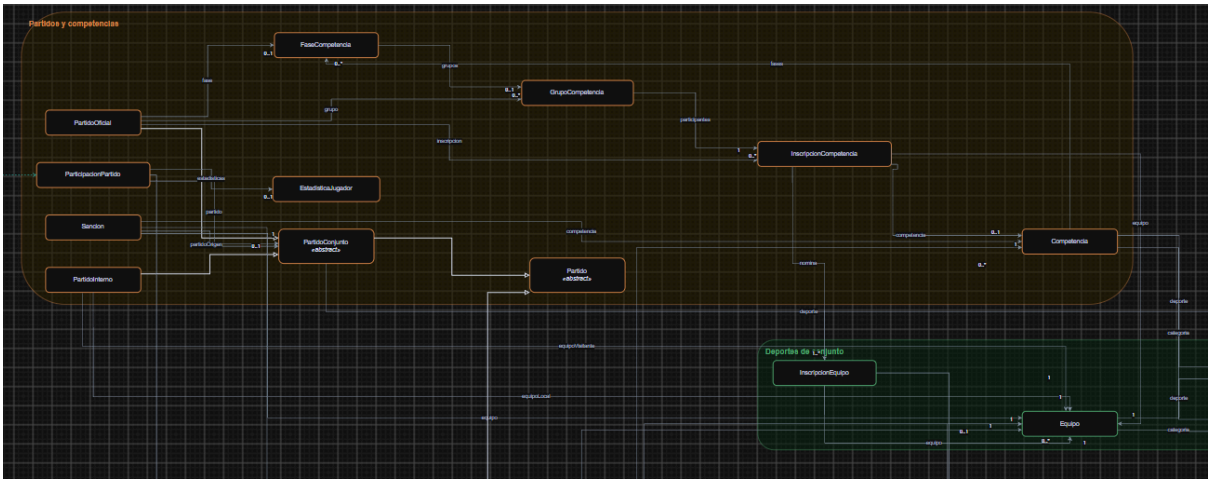
Persistencia: se encarga de guardar y recuperar la información del sistema mediante archivos.

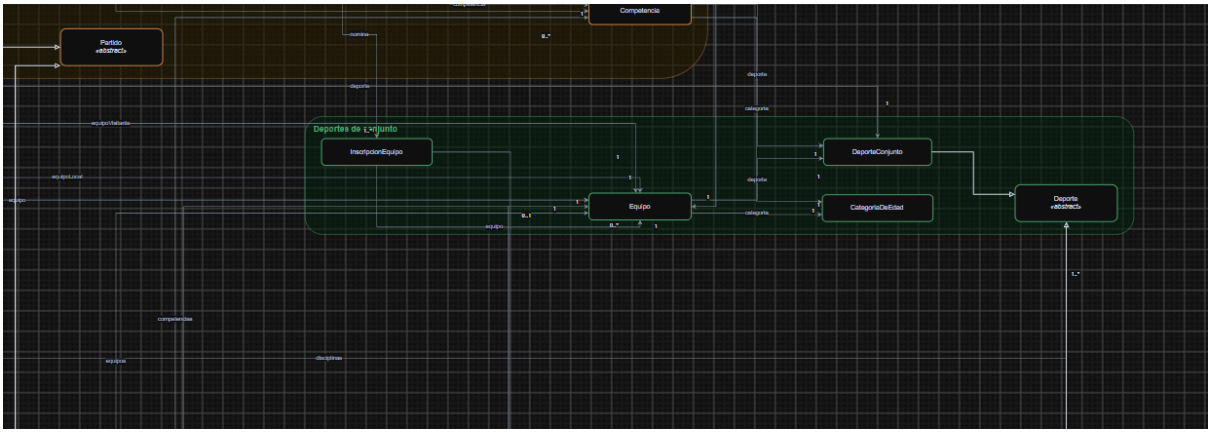
Estos componentes se relacionan de tal manera que ayudan a realizar las operaciones solicitadas. Las reglas del negocio que se validan dentro del modelo y los errores se comunican a cada instancia.

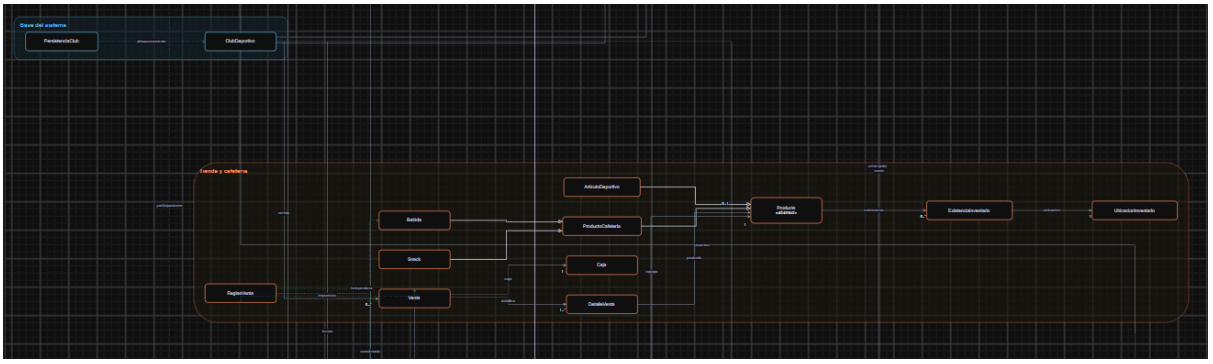
3. Diagrama de clases de alto nivel

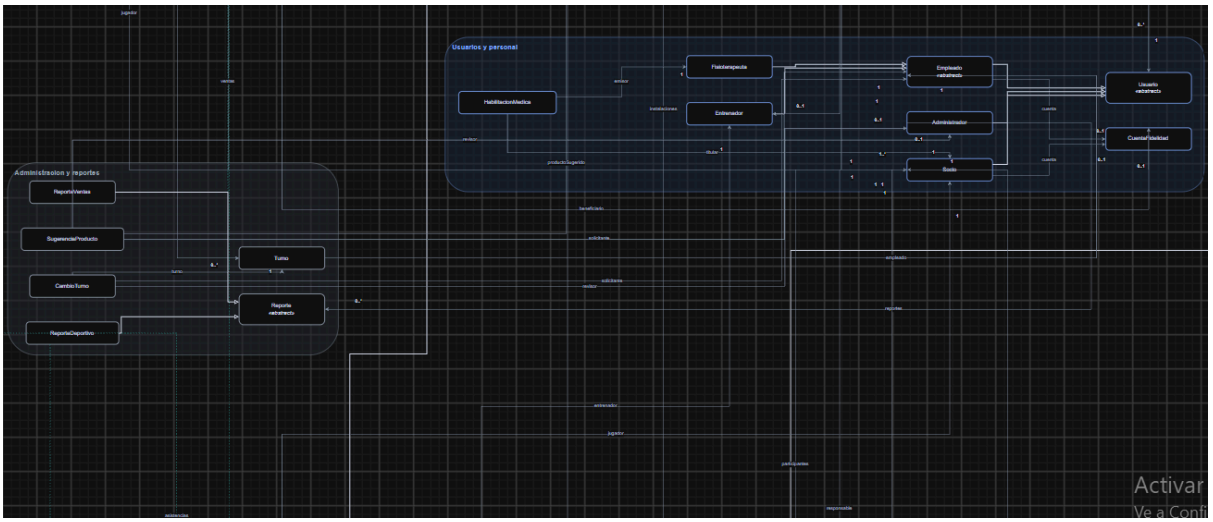
3.1 Descripción del modelo

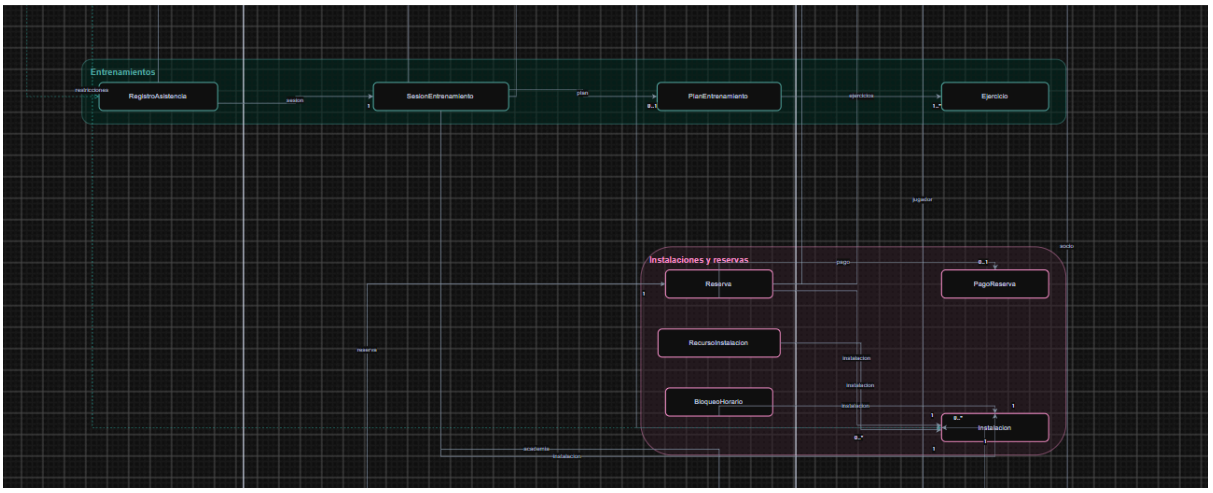
El diagrama de clases de alto nivel muestra la estructura general del sistema del Club Deportivo Séneca, incluyendo las clases principales y sus relaciones. Para facilitar su comprensión, se organizaron las clases según las diferentes áreas del club, manteniendo las asociaciones y relaciones de herencia necesarias para el funcionamiento del sistema.

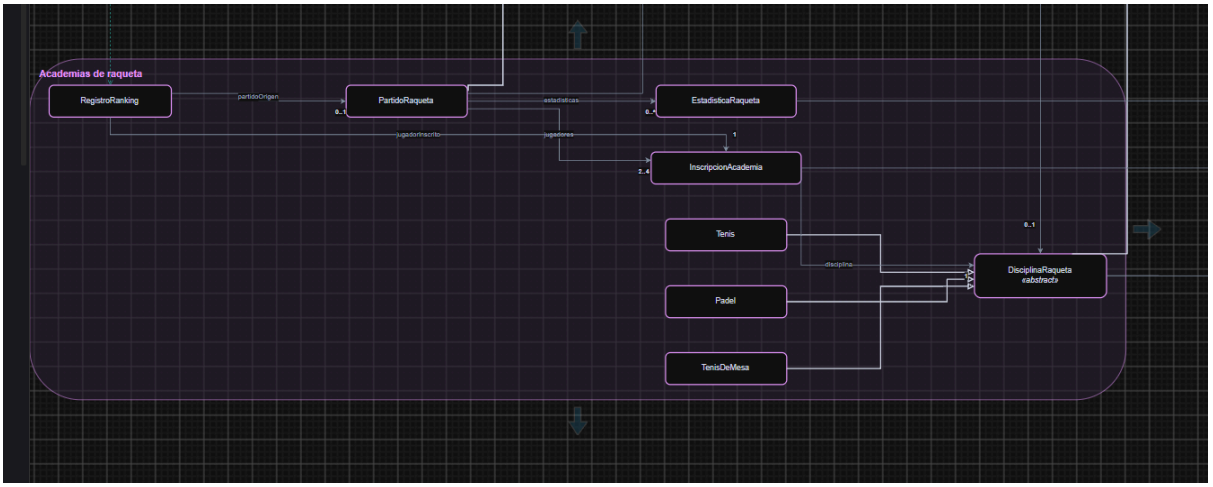




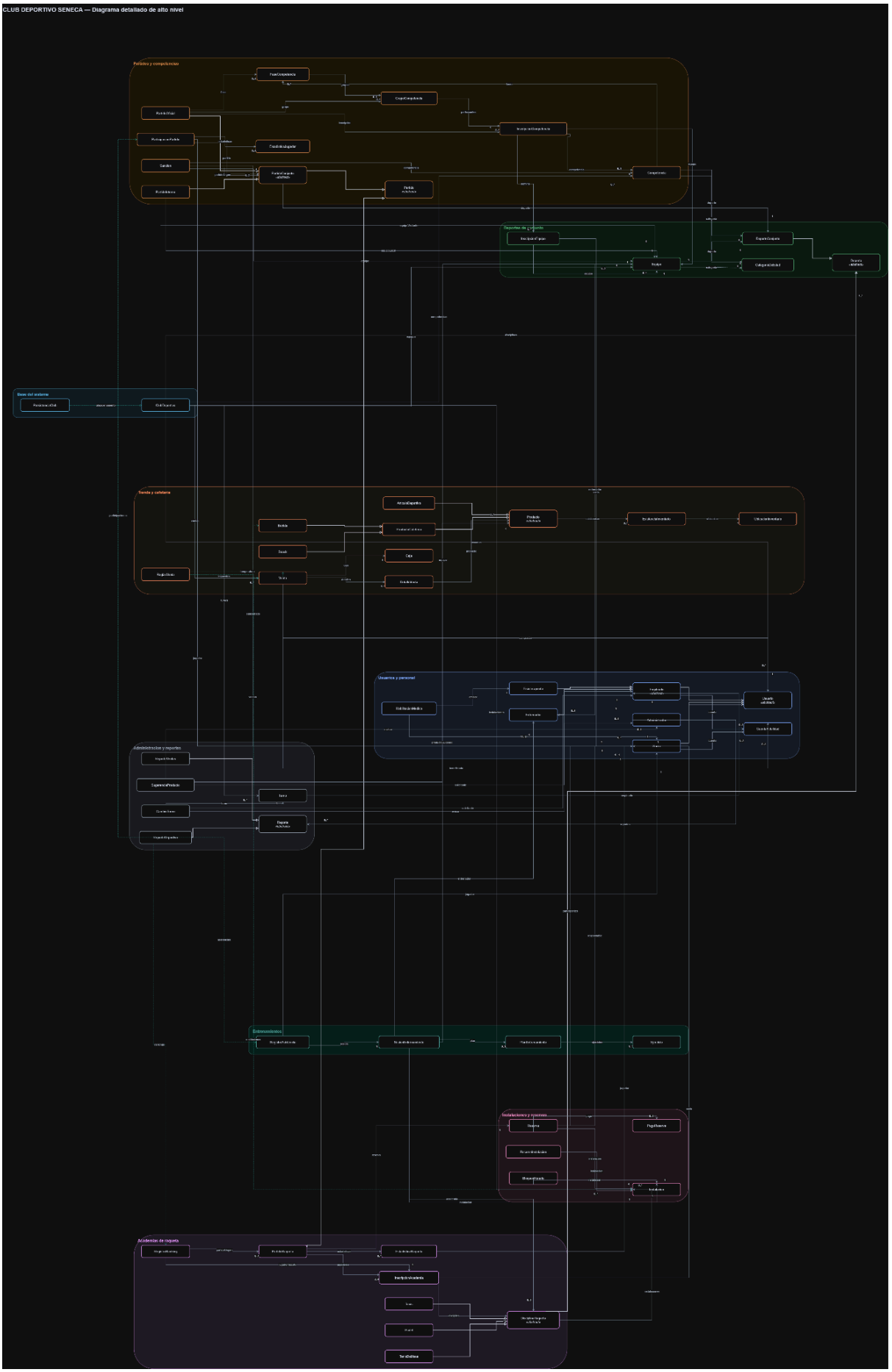



Usuarios y personal
Fisioterapeuta
Empleado
«abstracto»
Usuario
«abstracto»
HabilitacionMedica
Entrenador
Administrador
CuentaFidelidad
Socio
Administracion y reportes
ReporteVentas
SugerenciaProducto
Turno
CambioTurno
Reporte
«abstracto»
ReporteDeportivo
Activar
Ve a Confi


Entrenamientos
RegistroAsistencia
SesionEntrenamiento
PlanEntrenamiento
Ejercicio
Instalaciones y reservas
Reserva
PagoReserva
RecursoInstalacion
BloqueoHorario
Instalacion


Academias de raqueta
RegistroRanking
PartidoRaqueta
EstadisticaRaqueta
InscripcionAcademia
Tenis
Padel
TenisDeMesa
DisciplinaRaqueta
«abstract»

CLUB DEPORTIVO SENECA — Diagrama detallado de alto nivel



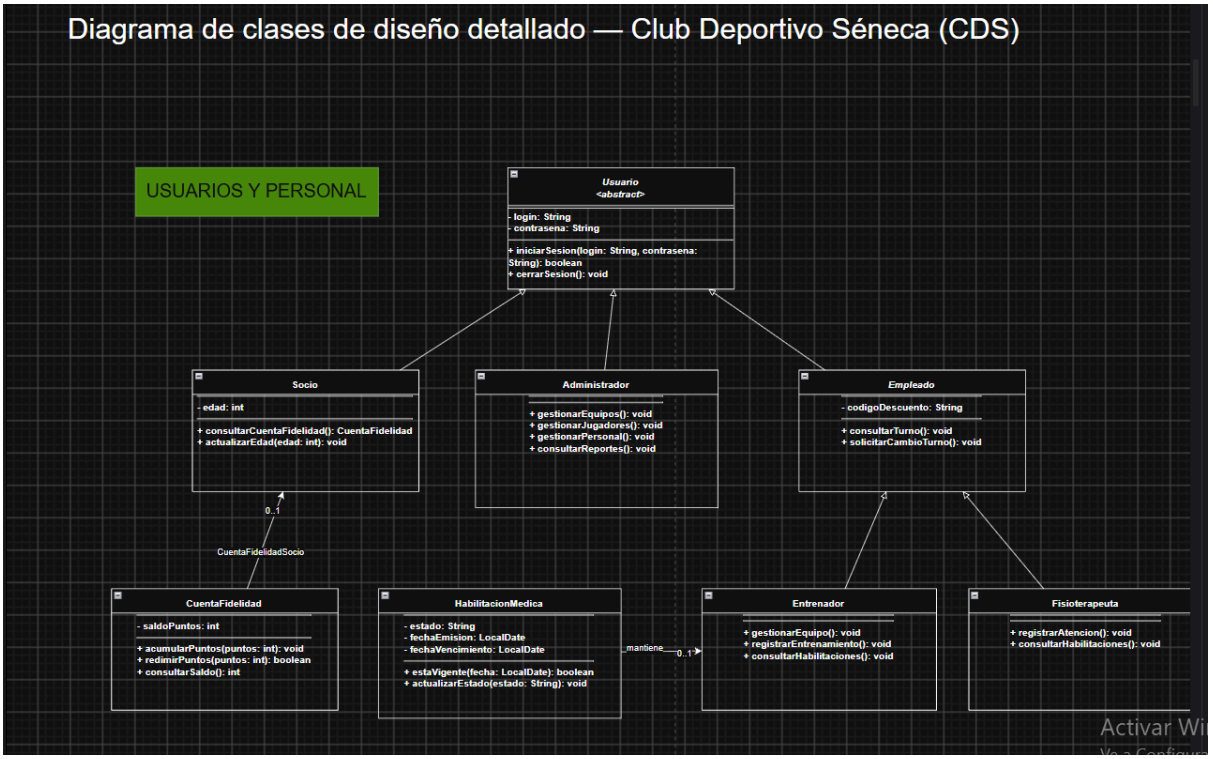
3.2 Organización de las clases

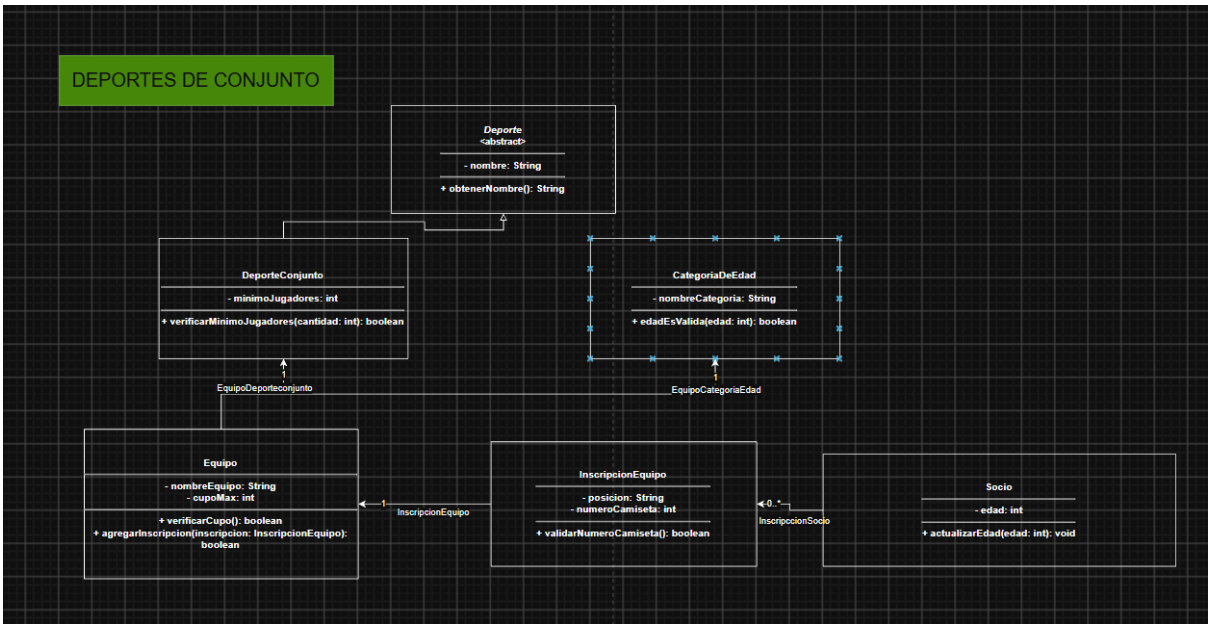
El modelo se encuentra organizado en diferentes grupos: usuarios y personal, deportes de conjunto, competencias, disciplinas individuales, entrenamientos, instalaciones, tienda, cafetería y administración. Esta división permite identificar las responsabilidades generales de cada área y las relaciones que existen entre ellas. Además, se utilizaron clases abstractas para agrupar características comunes y evitar repetir información entre clases que comparten comportamientos.

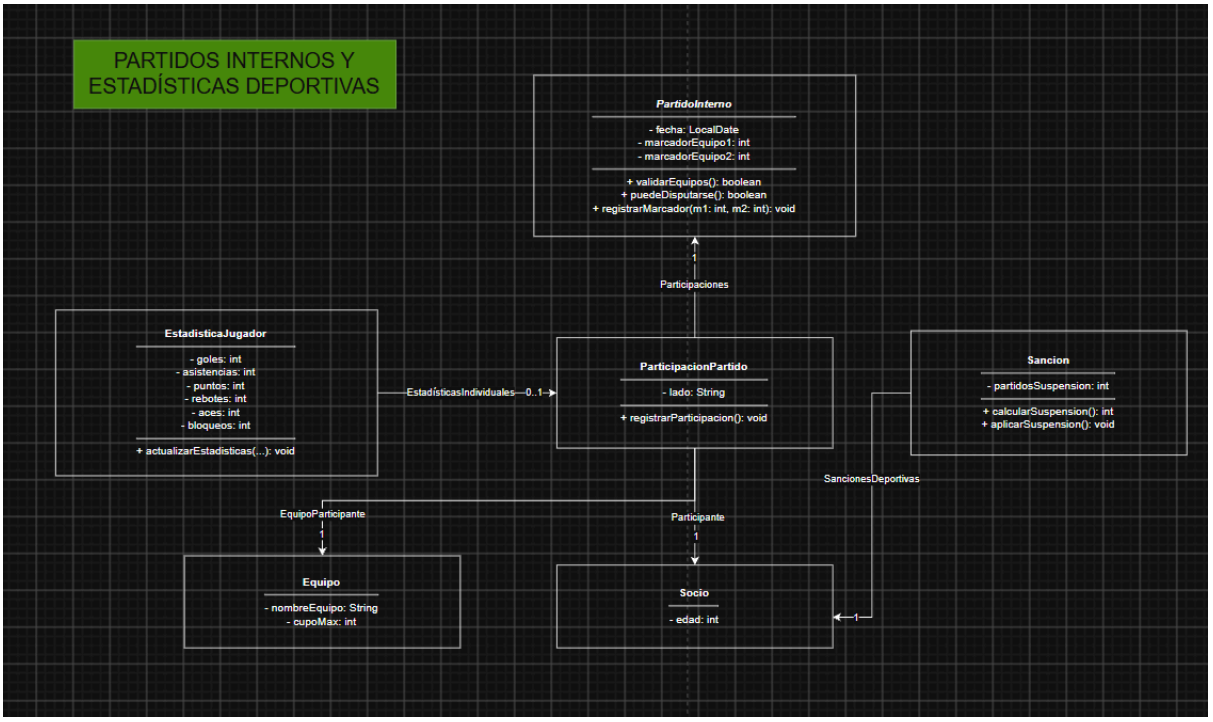
4. Diagrama de clases detallado

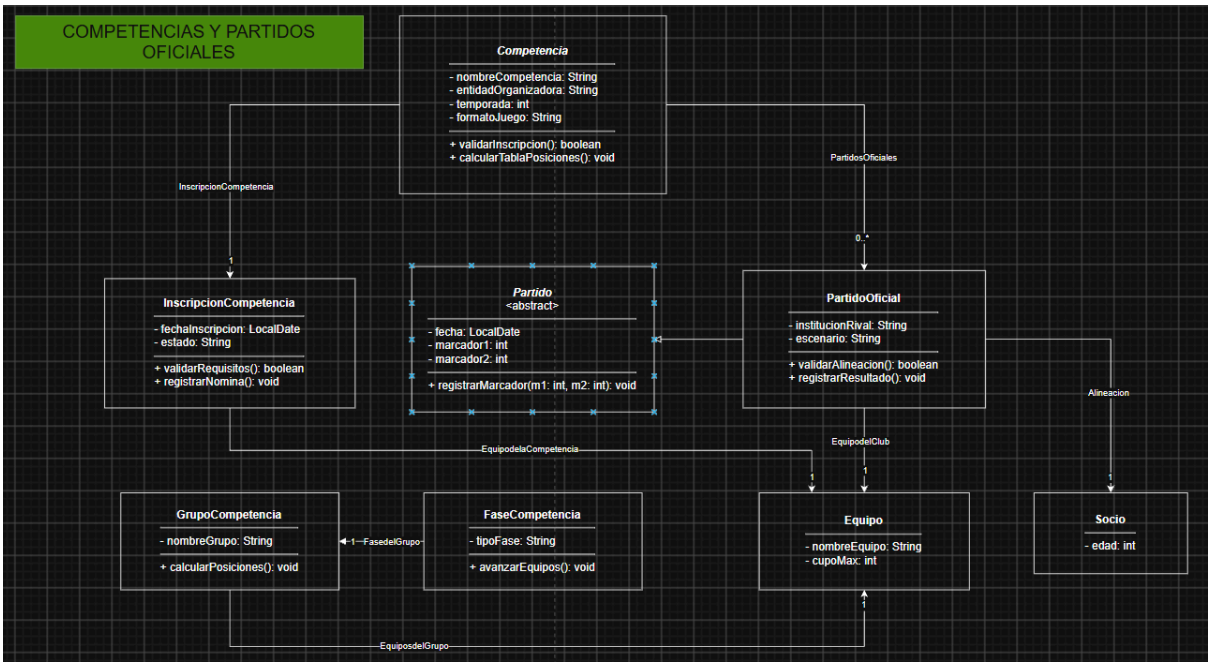
4.1 Descripción de Diseño

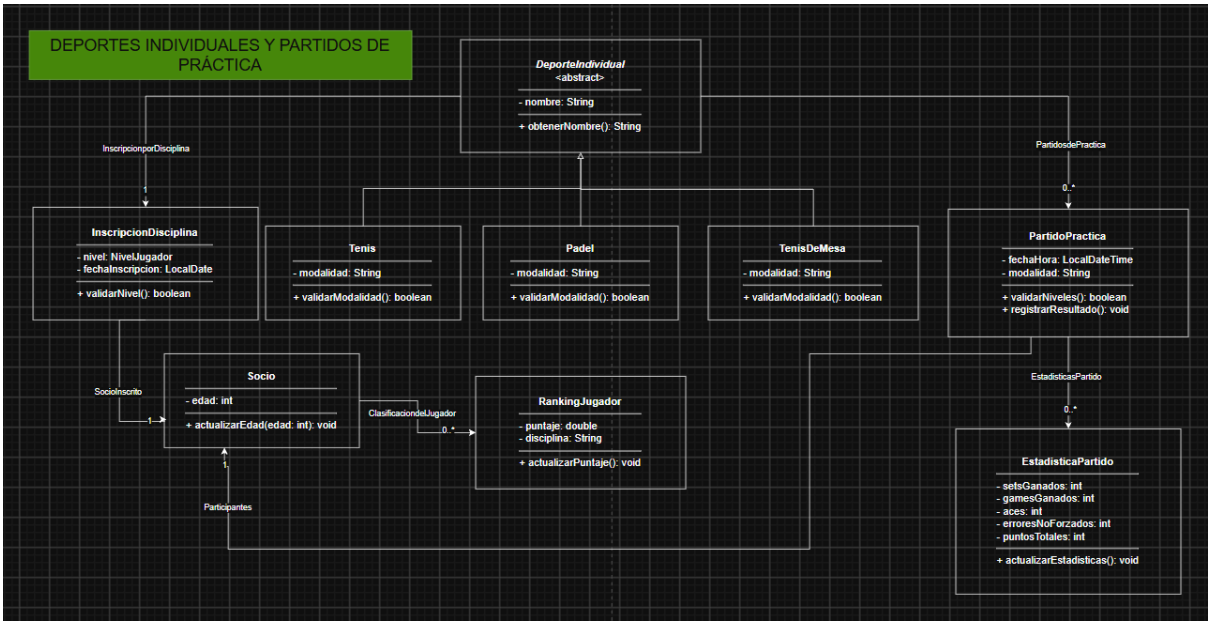
El diagrama de clases detallado desarrolla la estructura presentada en el modelo de alto nivel, incorporando los atributos, tipos de datos, métodos y relaciones de cada clase. Define elementos que se tuvieron en cuenta los requerimientos funcionales y las historias de usuario de la primera entrega. Las responsabilidades se distribuyeron entre las clases según la información que manejan y las operaciones que deben realizar. Además, se utilizaron clases abstractas y relaciones de herencia para compartir características comunes, mientras que las asociaciones permiten representar las interacciones entre los diferentes elementos del sistema.

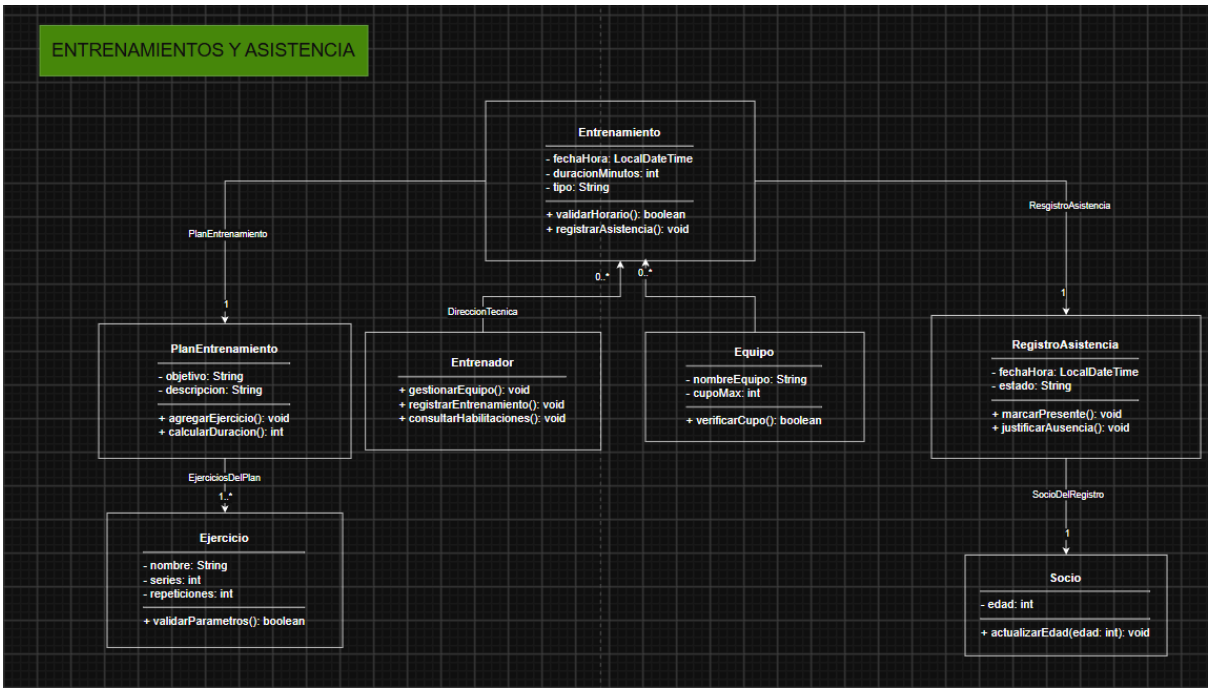


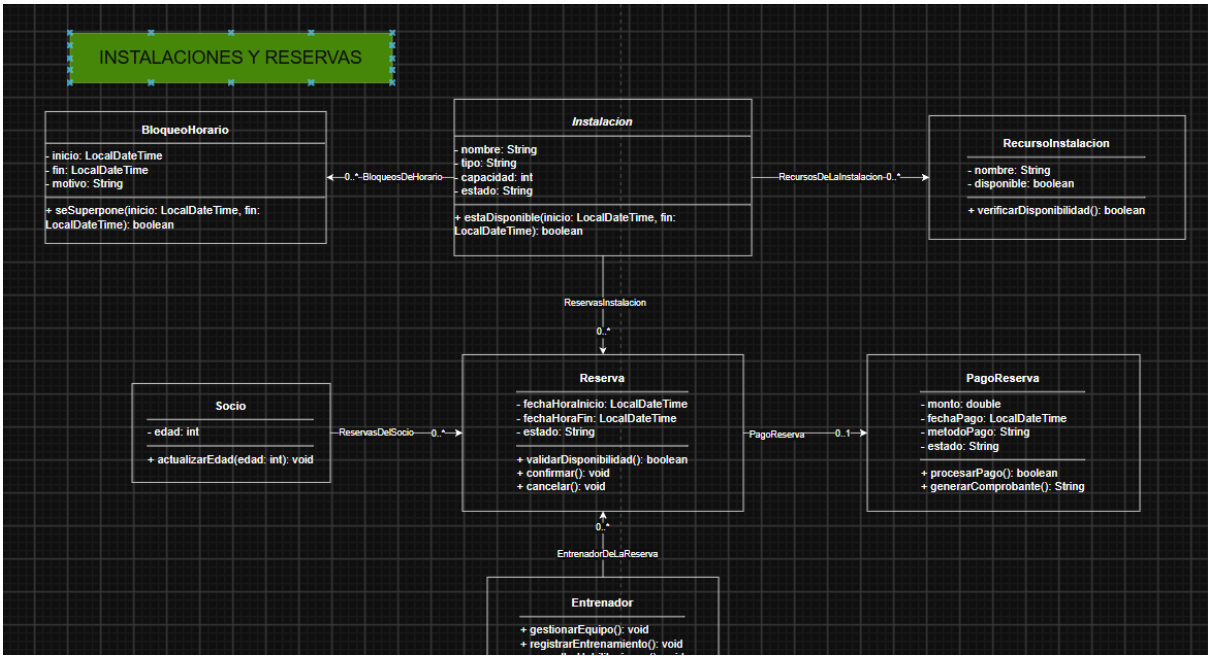

DEPORTES DE CONJUNTO
Deporte
<abstract>
- nombre: String
+ obtenerNombre(): String
DeporteConjunto
- minimoJugadores: int
+ verificarMinimoJugadores(cantidad: int): boolean
CategoriaDeEdad
- nombreCategoria: String
+ edadEsValida(edad: int): boolean
1
EquipoDeporteconjunto
1
EquipoCategoriaEdad
Equipo
- nombreEquipo: String
- cupoMax: int
+ verificarCupo(): boolean
+ agregarInscripcion(inscripcion: InscripcionEquipo): boolean
1
InscripcionEquipo
InscripcionEquipo
- posicion: String
- numeroCamiseta: int
+ validarNumeroCamiseta(): boolean
0..*
InscripcionSocio
Socio
- edad: int
+ actualizarEdad(edad: int): void

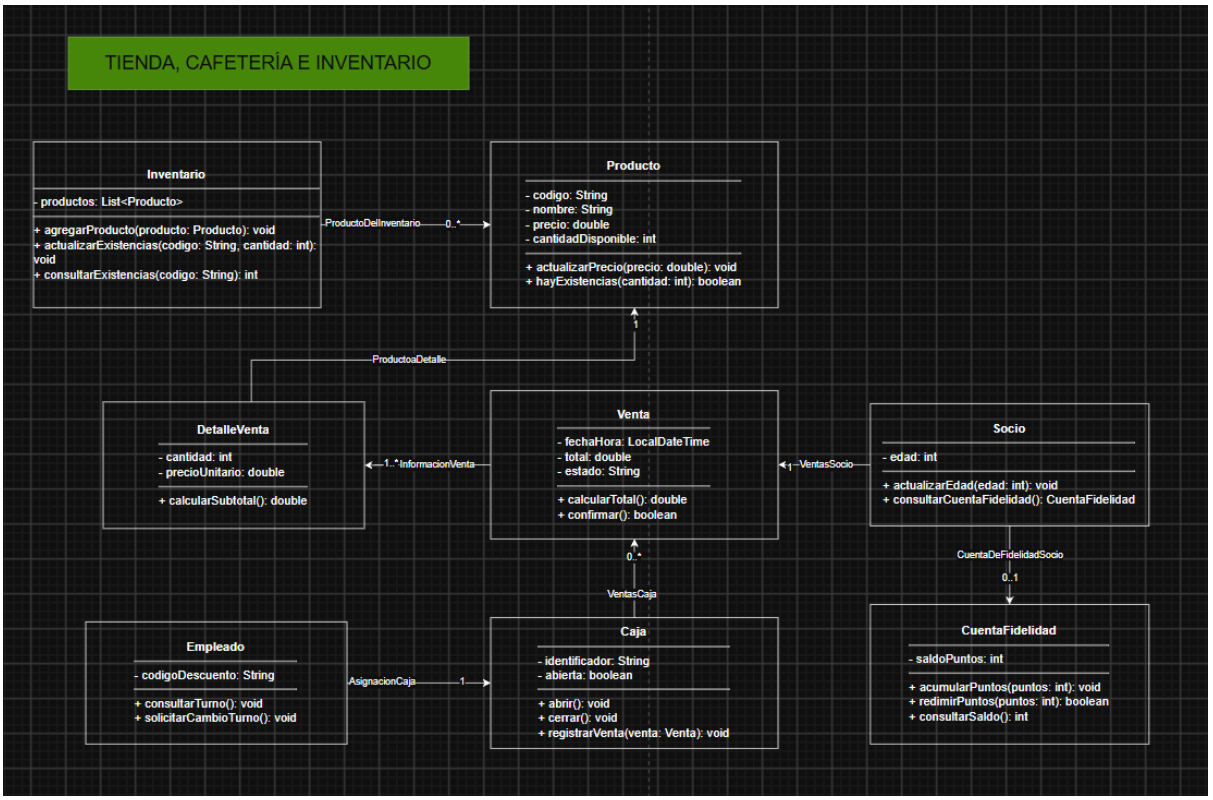

PARTIDOS INTERNOS Y ESTADÍSTICAS DEPORTIVAS
PartidoInterno
- fecha: LocalDate
- marcadorEquipo1: int
- marcadorEquipo2: int
+ validarEquipos(): boolean
+ puedeDisputarse(): boolean
+ registrarMarcador(m1: int, m2: int): void
1
Participaciones
EstadisticaJugador
- goles: int
- asistencias: int
- puntos: int
- rebotes: int
- aces: int
- bloqueos: int
+ actualizarEstadisticas(...): void
EstadísticasIndividuales
0..1
ParticipacionPartido
- lado: String
+ registrarParticipacion(): void
Sancion
- partidosSuspension: int
+ calcularSuspension(): int
+ aplicarSuspension(): void
SancionesDeportivas
EquipoParticipante
1
Participante
1
Equipo
- nombreEquipo: String
- cupoMax: int
Socio
- edad: int
1

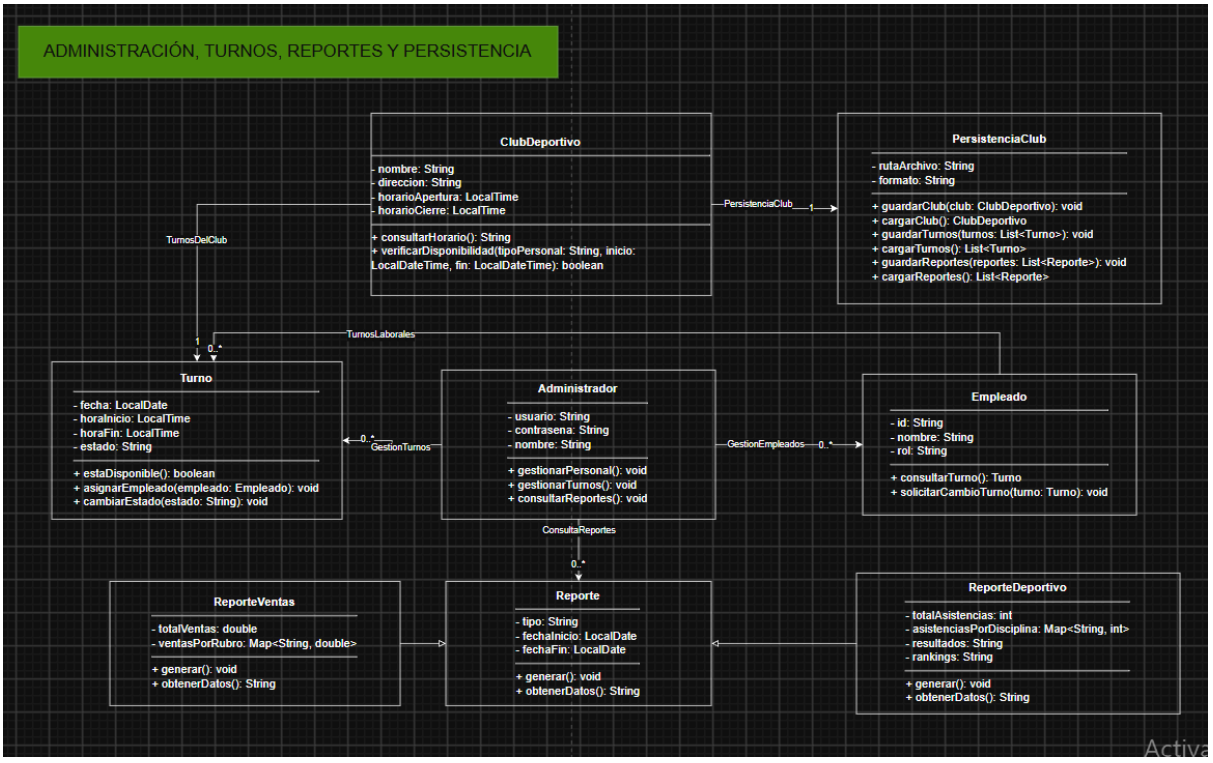

COMPETENCIAS Y PARTIDOS OFICIALES
Competencia
- nombreCompetencia: String
- entidadOrganizadora: String
- temporada: int
- formatoJuego: String
+ validarInscripcion(): boolean
+ calcularTablaPosiciones(): void
InscripcionCompetencia
PartidosOficiales
1
0..*
InscripcionCompetencia
- fechaInscripcion: LocalDate
- estado: String
+ validarRequisitos(): boolean
+ registrarNomina(): void
Partido
<abstract>
- fecha: LocalDate
- marcador1: int
- marcador2: int
+ registrarMarcador(m1: int, m2: int): void
PartidoOficial
- institucionRival: String
- escenario: String
+ validarAlineacion(): boolean
+ registrarResultado(): void
Alineacion
EquipodelaCompetencia
EquipodelClub
1
1
1
GrupoCompetencia
- nombreGrupo: String
+ calcularPosiciones(): void
1
FasedelGrupo
FaseCompetencia
- tipoFase: String
+ avanzarEquipos(): void
Equipo
- nombreEquipo: String
- cupoMax: int
Socio
- edad: int
1
EquiposdelGrupo

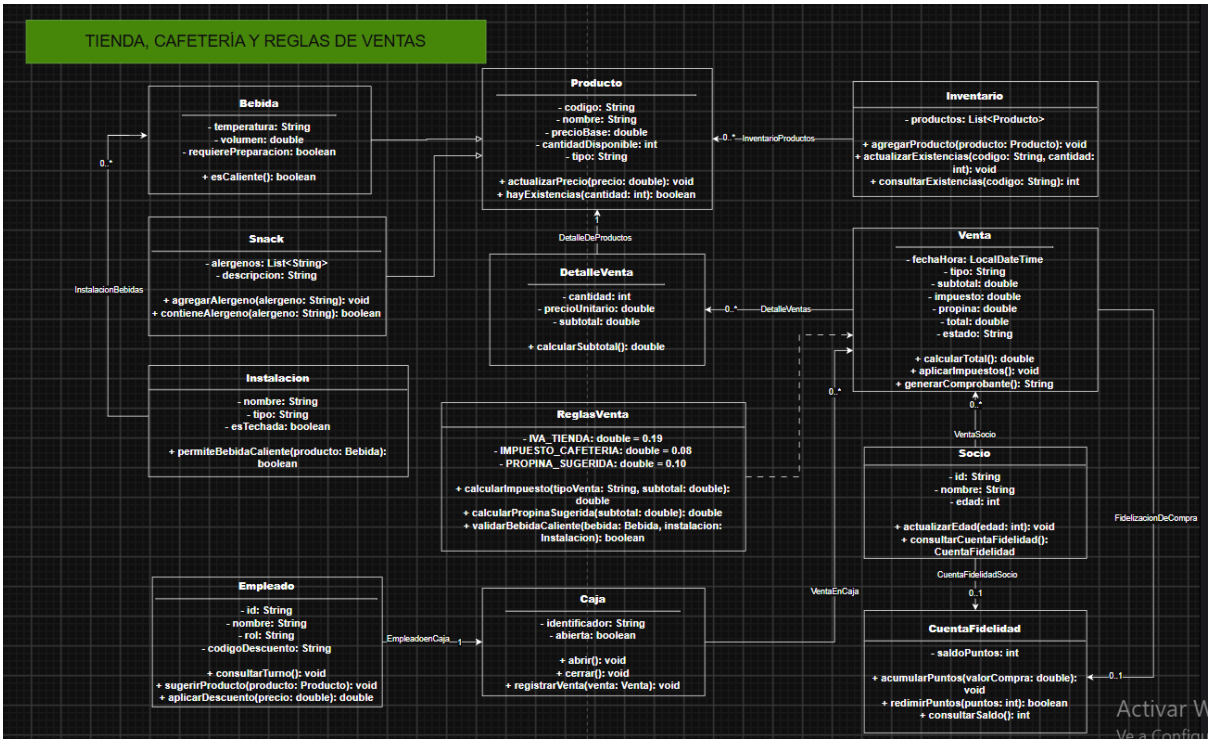

DEPORTES INDIVIDUALES Y PARTIDOS DE PRÁCTICA
DeporteIndividual
<abstract>
- nombre: String
+ obtenerNombre(): String
InscripcionporDisciplina
PartidosdePractica
1
0..*
InscripcionDisciplina
- nivel: NivelJugador
- fechaInscripcion: LocalDate
+ validarNivel(): boolean
Tenis
- modalidad: String
+ validarModalidad(): boolean
Padel
- modalidad: String
+ validarModalidad(): boolean
TenisDeMesa
- modalidad: String
+ validarModalidad(): boolean
PartidoPractica
- fechaHora: LocalDateTime
- modalidad: String
+ validarNiveles(): boolean
+ registrarResultado(): void
Socio
- edad: int
+ actualizarEdad(edad: int): void
SocioInscrito
1
ClasificaciondeJugador
0..*
RankingJugador
- puntaje: double
- disciplina: String
+ actualizarPuntaje(): void
EstadisticasPartido
0..*
1
Participantes
EstadisticaPartido
- setsGanados: int
- gamesGanados: int
- aces: int
- erroresNoForzados: int
- puntosTotales: int
+ actualizarEstadisticas(): void


ENTRENAMIENTOS Y ASISTENCIA
Entrenamiento
- fechaHora: LocalDateTime
- duracionMinutos: int
- tipo: String
+ validarHorario(): boolean
+ registrarAsistencia(): void
PlanEntrenamiento
1
RegistroAsistencia
1
0..*
0..*
DireccionTecnica
PlanEntrenamiento
- objetivo: String
- descripcion: String
+ agregarEjercicio(): void
+ calcularDuracion(): int
Entrenador
+ gestionarEquipo(): void
+ registrarEntrenamiento(): void
+ consultarHabilitaciones(): void
Equipo
- nombreEquipo: String
- cupoMax: int
+ verificarCupo(): boolean
RegistroAsistencia
- fechaHora: LocalDateTime
- estado: String
+ marcarPresente(): void
+ justificarAusencia(): void
EjerciciosDelPlan
1..*
SocioDelRegistro
1
Ejercicio
- nombre: String
- series: int
- repeticiones: int
+ validarParametros(): boolean
Socio
- edad: int
+ actualizarEdad(edad: int): void


INSTALACIONES Y RESERVAS
BloqueoHorario
- inicio: LocalDateTime
- fin: LocalDateTime
- motivo: String
+ seSuperpone(inicio: LocalDateTime, fin: LocalDateTime): boolean
0..*
BloqueosDeHorario
Instalacion
- nombre: String
- tipo: String
- capacidad: int
- estado: String
+ estaDisponible(inicio: LocalDateTime, fin: LocalDateTime): boolean
RecursosDeLaInstalacion
0..*
RecursoInstalacion
- nombre: String
- disponible: boolean
+ verificarDisponibilidad(): boolean
ReservasInstalacion
0..*
Socio
- edad: int
+ actualizarEdad(edad: int): void
ReservasDelSocio
0..*
Reserva
- fechaHoraInicio: LocalDateTime
- fechaHoraFin: LocalDateTime
- estado: String
+ validarDisponibilidad(): boolean
+ confirmar(): void
+ cancelar(): void
PagoReserva
0..1
PagoReserva
- monto: double
- fechaPago: LocalDateTime
- metodoPago: String
- estado: String
+ procesarPago(): boolean
+ generarComprobante(): String
0..*
EntrenadorDeLaReserva
Entrenador
+ gestionarEquipo(): void
+ registrarEntrenamiento(): void


TIENDA, CAFETERÍA E INVENTARIO
Inventario
- productos: List<Producto>
+ agregarProducto(producto: Producto): void
+ actualizarExistencias(codigo: String, cantidad: int): void
+ consultarExistencias(codigo: String): int
ProductoDelInventario
0..*
Producto
- codigo: String
- nombre: String
- precio: double
- cantidadDisponible: int
+ actualizarPrecio(precio: double): void
+ hayExistencias(cantidad: int): boolean
1
ProductoaDetalle
DetalleVenta
- cantidad: int
- precioUnitario: double
+ calcularSubtotal(): double
1..*
InformacionVenta
Venta
- fechaHora: LocalDateTime
- total: double
- estado: String
+ calcularTotal(): double
+ confirmar(): boolean
1
VentasSocio
Socio
- edad: int
+ actualizarEdad(edad: int): void
+ consultarCuentaFidelidad(): CuentaFidelidad
CuentaDeFidelidadSocio
0..1
0..*
VentasCaja
Empleado
- codigoDescuento: String
+ consultarTurno(): void
+ solicitarCambioTurno(): void
AsignacionCaja
1
Caja
- identificador: String
- abierta: boolean
+ abrir(): void
+ cerrar(): void
+ registrarVenta(venta: Venta): void
CuentaFidelidad
- saldoPuntos: int
+ acumularPuntos(puntos: int): void
+ redimirPuntos(puntos: int): boolean
+ consultarSaldo(): int


ADMINISTRACIÓN, TURNOS, REPORTES Y PERSISTENCIA
ClubDeportivo
- nombre: String
- direccion: String
- horarioApertura: LocalTime
- horarioCierre: LocalTime
+ consultarHorario(): String
+ verificarDisponibilidad(tipoPersonal: String, inicio: LocalDateTime, fin: LocalDateTime): boolean
PersistenciaClub
- rutaArchivo: String
- formato: String
+ guardarClub(club: ClubDeportivo): void
+ cargarClub(): ClubDeportivo
+ guardarTurnos(turnos: List<Turno>): void
+ cargarTurnos(): List<Turno>
+ guardarReportes(reportes: List<Reporte>): void
+ cargarReportes(): List<Reporte>
PersistenciaClub
1
TurnosDelClub
1
0..*
TurnosLaborales
Turno
- fecha: LocalDate
- horaInicio: LocalTime
- horaFin: LocalTime
- estado: String
+ estaDisponible(): boolean
+ asignarEmpleado(empleado: Empleado): void
+ cambiarEstado(estado: String): void
0..*
GestionTurnos
Administrador
- usuario: String
- contrasena: String
- nombre: String
+ gestionarPersonal(): void
+ gestionarTurnos(): void
+ consultarReportes(): void
GestionEmpleados
0..*
Empleado
- id: String
- nombre: String
- rol: String
+ consultarTurno(): Turno
+ solicitarCambioTurno(turno: Turno): void
ConsultaReportes
0..*
ReporteVentas
- totalVentas: double
- ventasPorRubro: Map<String, double>
+ generar(): void
+ obtenerDatos(): String
Reporte
- tipo: String
- fechaInicio: LocalDate
- fechaFin: LocalDate
+ generar(): void
+ obtenerDatos(): String
ReporteDeportivo
- totalAsistencias: int
- asistenciasPorDisciplina: Map<String, int>
- resultados: String
- rankings: String
+ generar(): void
+ obtenerDatos(): String

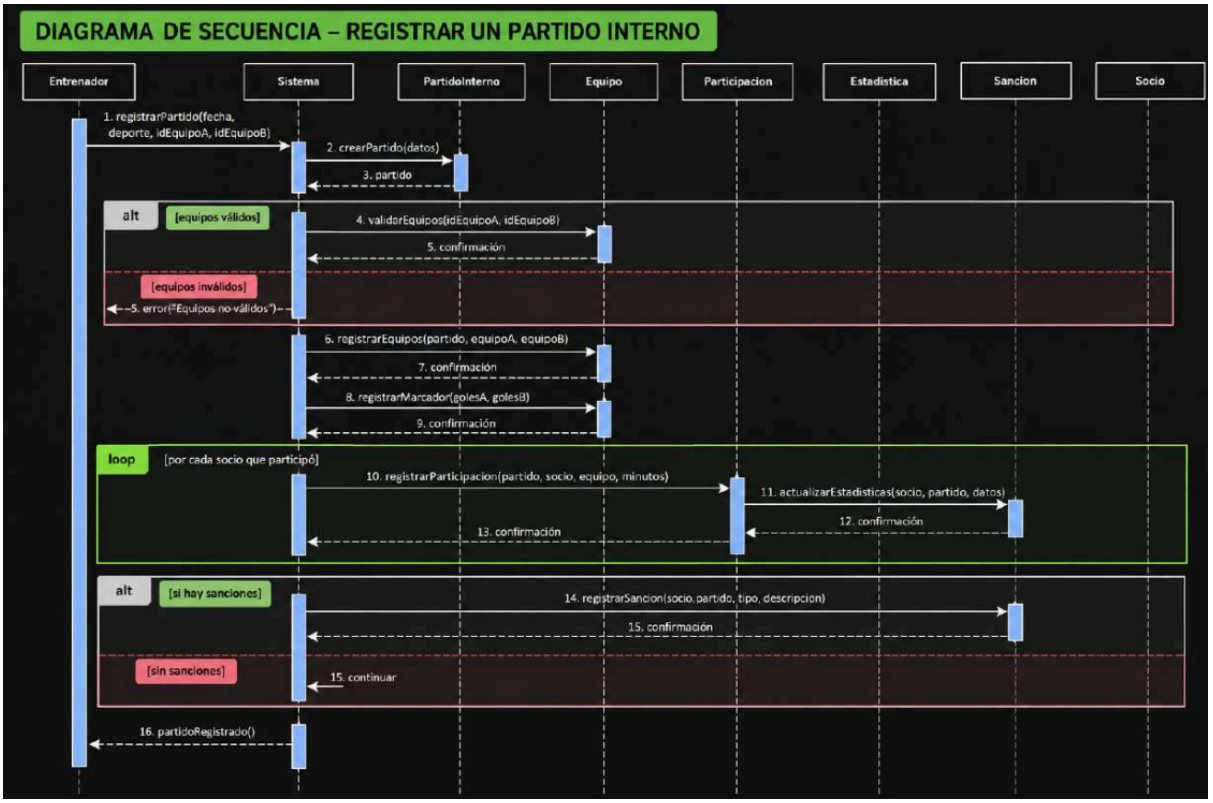

TIENDA, CAFETERÍA Y REGLAS DE VENTAS
Bebida
- temperatura: String
- volumen: double
- requierePreparacion: boolean
+ esCaliente(): boolean
Producto
- codigo: String
- nombre: String
- precioBase: double
- cantidadDisponible: int
- tipo: String
+ actualizarPrecio(precio: double): void
+ hayExistencias(cantidad: int): boolean
0..*
InventarioProductos
Inventario
- productos: List<Producto>
+ agregarProducto(producto: Producto): void
+ actualizarExistencias(codigo: String, cantidad: int): void
+ consultarExistencias(codigo: String): int
0..*
InstalacionBebidas
Snack
- alergenos: List<String>
- descripcion: String
+ agregarAlergeno(alergeno: String): void
+ contieneAlergeno(alergeno: String): boolean
1
DetalleDeProductos
DetalleVenta
- cantidad: int
- precioUnitario: double
- subtotal: double
+ calcularSubtotal(): double
0..*
DetalleVentas
Venta
- fechaHora: LocalDateTime
- tipo: String
- subtotal: double
- impuesto: double
- propina: double
- total: double
- estado: String
+ calcularTotal(): double
+ aplicarImpuestos(): void
+ generarComprobante(): String
Instalacion
- nombre: String
- tipo: String
- esTechada: boolean
+ permiteBebidaCaliente(producto: Bebida): boolean
ReglasVenta
- IVA_TIENDA: double = 0.19
- IMPUESTO_CAFETERIA: double = 0.08
- PROPINA_SUGERIDA: double = 0.10
+ calcularImpuesto(tipoVenta: String, subtotal: double): double
+ calcularPropinaSugerida(subtotal: double): double
+ validarBebidaCaliente(bebida: Bebida, instalacion: Instalacion): boolean
0..*
VentaSocio
Socio
- id: String
- nombre: String
- edad: int
+ actualizarEdad(edad: int): void
+ consultarCuentaFidelidad(): CuentaFidelidad
FidelizacionDeCompra
CuentaFidelidadSocio
0..1
Empleado
- id: String
- nombre: String
- rol: String
- codigoDescuento: String
+ consultarTurno(): void
+ sugerirProducto(producto: Producto): void
+ aplicarDescuento(precio: double): double
EmpleadoenCaja
1
Caja
- identificador: String
- abierta: boolean
+ abrir(): void
+ cerrar(): void
+ registrarVenta(venta: Venta): void
VentaEnCaja
CuentaFidelidad
- saldoPuntos: int
+ acumularPuntos(valorCompra: double): void
+ redimirPuntos(puntos: int): boolean
+ consultarSaldo(): int
0..1

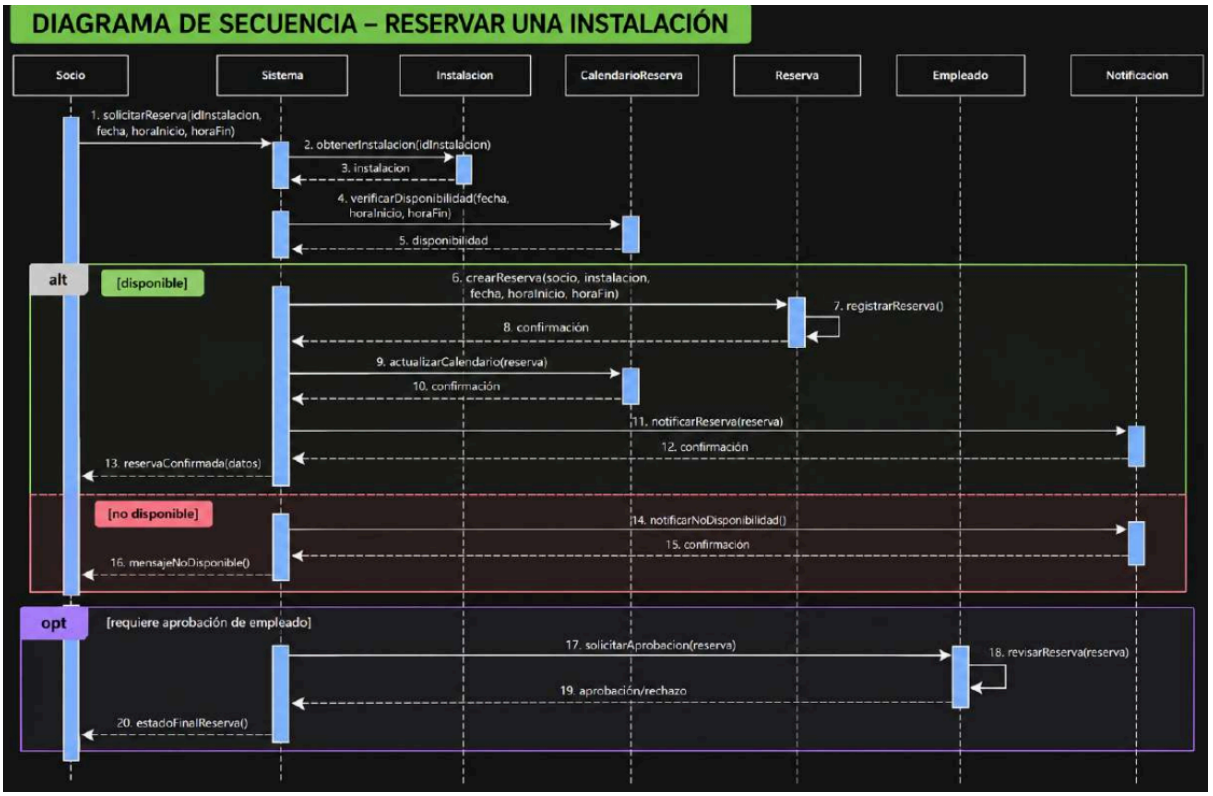
5. Diagramas de secuencia

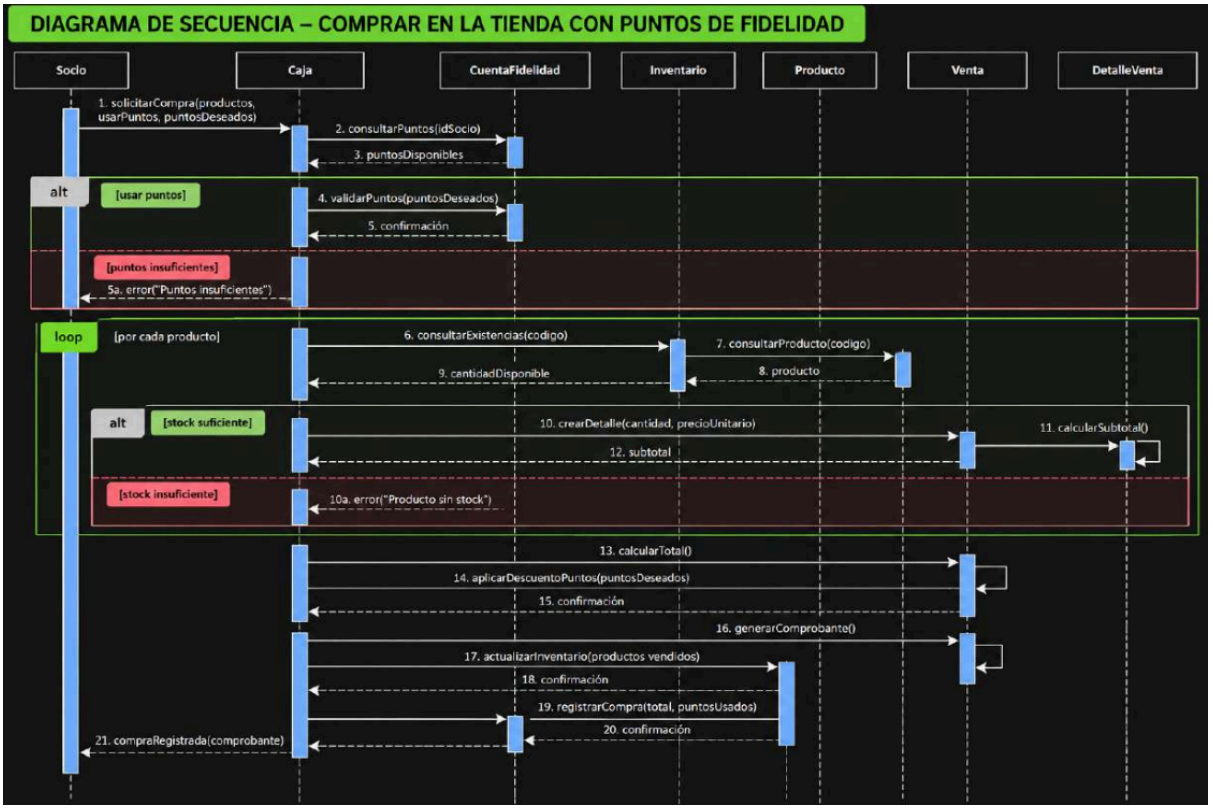
5.1 Descripción

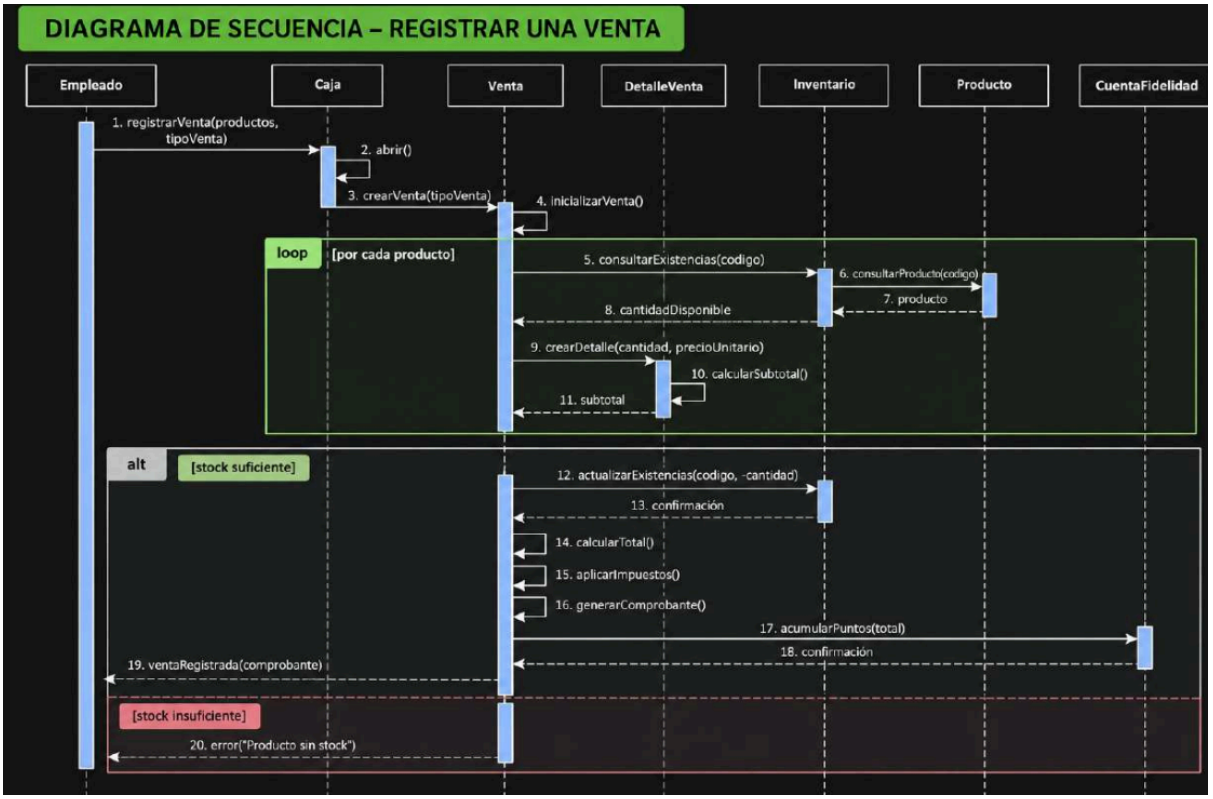
Los diagramas de secuencia permiten representar cómo interactúan las diferentes clases del sistema para realizar una funcionalidad. Para esta sección se seleccionan algunas de las operaciones más importantes del Club Deportivo Séneca, teniendo en cuenta las validaciones necesarias y las responsabilidades de cada clase. Estos diagramas muestran el orden en que se realizan las operaciones, los mensajes entre objetos y las condiciones que pueden permitir o impedir que una acción se complete.

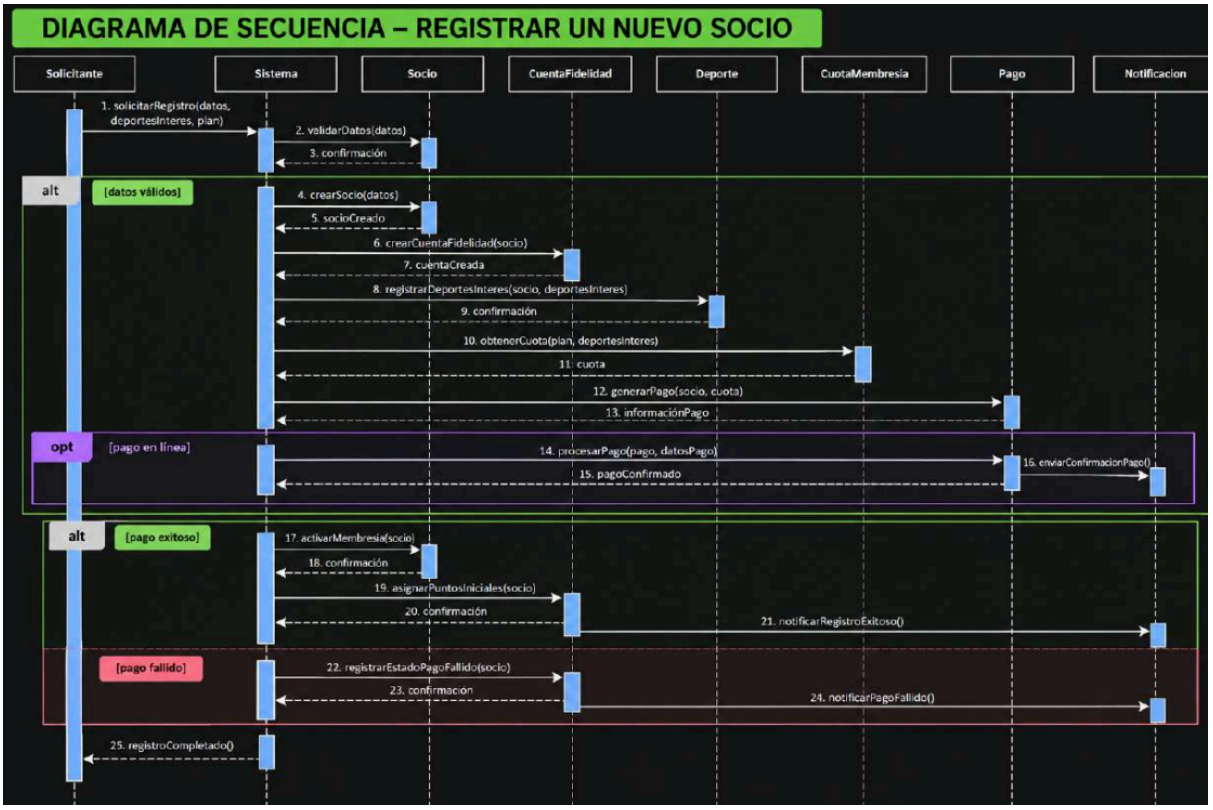
5.2 Funcionalidades criticas de los diagramas de secuencia

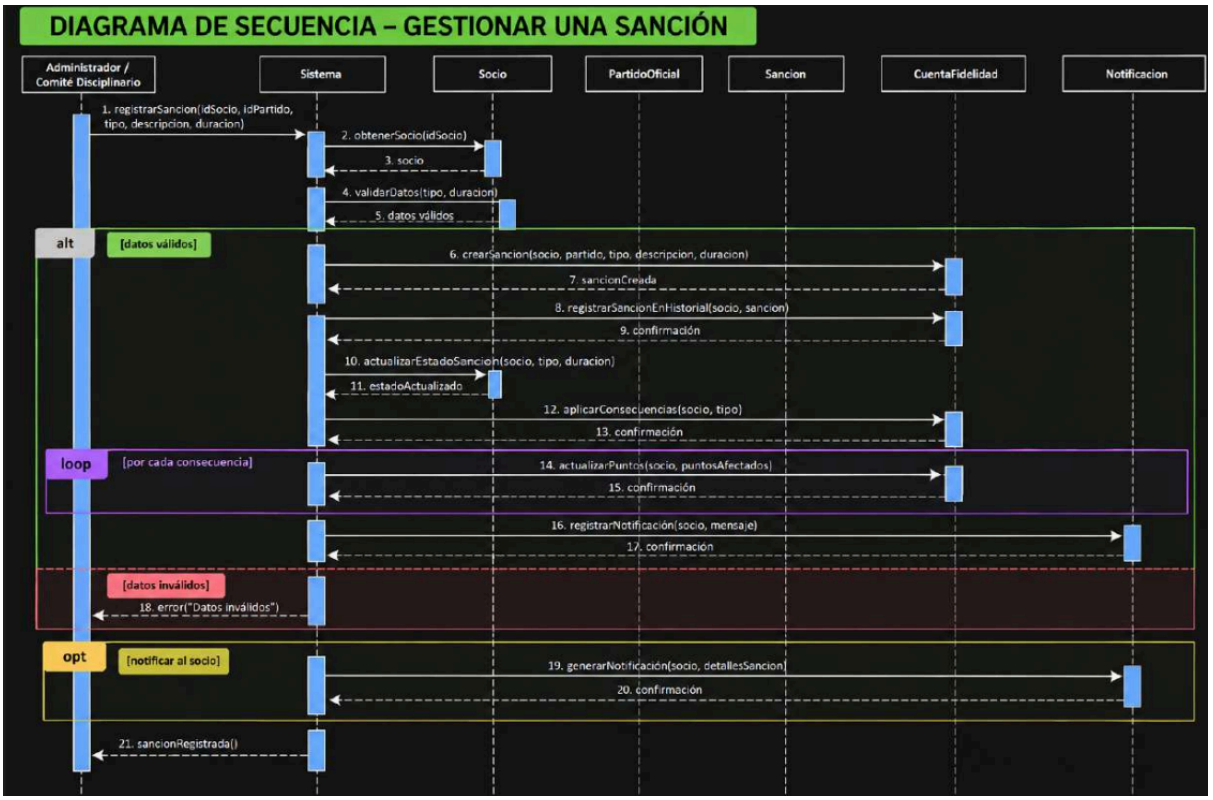


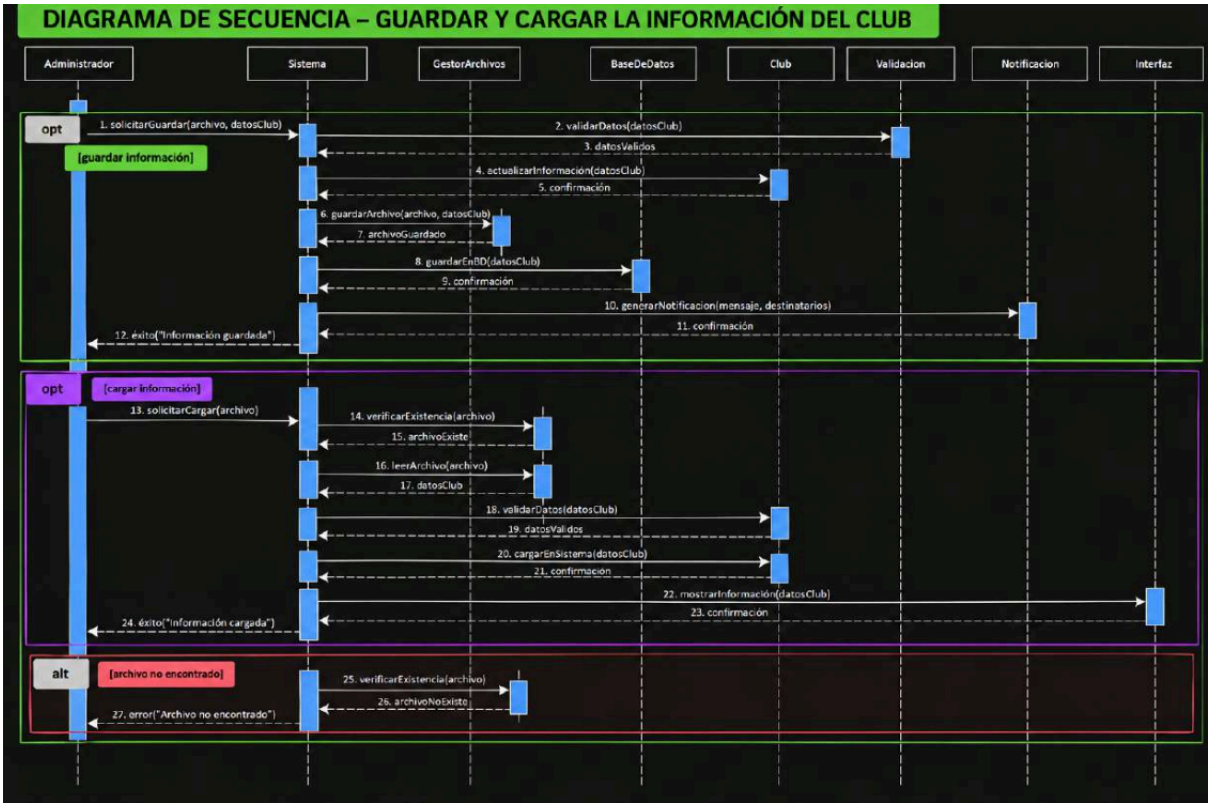

DIAGRAMA DE SECUENCIA – RESERVAR UNA INSTALACIÓN
Socio
Sistema
Instalacion
CalendarioReserva
Reserva
Empleado
Notificacion
1. solicitarReserva(idInstalacion, fecha, horaInicio, horaFin)
2. obtenerInstalacion(idInstalacion)
3. instalacion
4. verificarDisponibilidad(fecha, horaInicio, horaFin)
5. disponibilidad
alt
[disponible]
6. crearReserva(socio, instalacion, fecha, horaInicio, horaFin)
7. registrarReserva()
8. confirmación
9. actualizarCalendario(reserva)
10. confirmación
11. notificarReserva(reserva)
12. confirmación
13. reservaConfirmada(datos)
[no disponible]
14. notificarNoDisponibilidad()
15. confirmación
16. mensajeNoDisponible()
opt
[requiere aprobación de empleado]
17. solicitarAprobacion(reserva)
18. revisarReserva(reserva)
19. aprobación/rechazo
20. estadoFinalReserva()


DIAGRAMA DE SECUENCIA – COMPRAR EN LA TIENDA CON PUNTOS DE FIDELIDAD
Socio
Caja
CuentaFidelidad
Inventario
Producto
Venta
DetalleVenta
1. solicitarCompra(productos, usarPuntos, puntosDeseados)
2. consultarPuntos(idSocio)
3. puntosDisponibles
alt
[usar puntos]
4. validarPuntos(puntosDeseados)
5. confirmación
[puntos insuficientes]
5a. error("Puntos insuficientes")
loop
[por cada producto]
6. consultarExistencias(codigo)
7. consultarProducto(codigo)
8. producto
9. cantidadDisponible
alt
[stock suficiente]
10. crearDetalle(cantidad, precioUnitario)
11. calcularSubtotal()
12. subtotal
[stock insuficiente]
10a. error("Producto sin stock")
13. calcularTotal()
14. aplicarDescuentoPuntos(puntosDeseados)
15. confirmación
16. generarComprobante()
17. actualizarInventario(productos vendidos)
18. confirmación
19. registrarCompra(total, puntosUsados)
20. confirmación
21. compraRegistrada(comprobante)


DIAGRAMA DE SECUENCIA – REGISTRAR UNA VENTA
Empleado
Caja
Venta
DetalleVenta
Inventario
Producto
CuentaFidelidad
1. registrarVenta(productos, tipoVenta)
2. abrir()
3. crearVenta(tipoVenta)
4. inicializarVenta()
loop
[por cada producto]
5. consultarExistencias(codigo)
6. consultarProducto(codigo)
7. producto
8. cantidadDisponible
9. crearDetalle(cantidad, precioUnitario)
10. calcularSubtotal()
11. subtotal
alt
[stock suficiente]
12. actualizarExistencias(codigo, -cantidad)
13. confirmación
14. calcularTotal()
15. aplicarImpuestos()
16. generarComprobante()
17. acumularPuntos(total)
18. confirmación
19. ventaRegistrada(comprobante)
[stock insuficiente]
20. error("Producto sin stock")


DIAGRAMA DE SECUENCIA – REGISTRAR UN NUEVO SOCIO
Solicitante
Sistema
Socio
CuentaFidelidad
Deporte
CuotaMembresia
Pago
Notificacion
1. solicitarRegistro(datos, deportesInteres, plan)
2. validarDatos(datos)
3. confirmación
alt
[datos válidos]
4. crearSocio(datos)
5. socioCreado
6. crearCuentaFidelidad(socio)
7. cuentaCreada
8. registrarDeportesInteres(socio, deportesInteres)
9. confirmación
10. obtenerCuota(plan, deportesInteres)
11. cuota
12. generarPago(socio, cuota)
13. informaciónPago
opt
[pago en línea]
14. procesarPago(pago, datosPago)
15. pagoConfirmado
16. enviarConfirmacionPago()
alt
[pago exitoso]
17. activarMembresia(socio)
18. confirmación
19. asignarPuntosIniciales(socio)
20. confirmación
21. notificarRegistroExitoso()
[pago fallido]
22. registrarEstadoPagoFallido(socio)
23. confirmación
24. notificarPagoFallido()
25. registroCompletado()


DIAGRAMA DE SECUENCIA – GESTIONAR UNA SANCIÓN
Administrador / Comité Disciplinario
Sistema
Socio
PartidoOficial
Sancion
CuentaFidelidad
Notificacion
1. registrarSancion(idSocio, idPartido, tipo, descripcion, duracion)
2. obtenerSocio(idSocio)
3. socio
4. validarDatos(tipo, duracion)
5. datos válidos
alt
[datos válidos]
6. crearSancion(socio, partido, tipo, descripcion, duracion)
7. sancionCreada
8. registrarSancionEnHistorial(socio, sancion)
9. confirmación
10. actualizarEstadoSancion(socio, tipo, duracion)
11. estadoActualizado
12. aplicarConsecuencias(socio, tipo)
13. confirmación
loop
[por cada consecuencia]
14. actualizarPuntos(socio, puntosAfectados)
15. confirmación
16. registrarNotificación(socio, mensaje)
17. confirmación
[datos inválidos]
18. error("Datos inválidos")
opt
[notificar al socio]
19. generarNotificación(socio, detallesSancion)
20. confirmación
21. sancionRegistrada()


DIAGRAMA DE SECUENCIA – GUARDAR Y CARGAR LA INFORMACIÓN DEL CLUB
Administrador
Sistema
GestorArchivos
BaseDeDatos
Club
Validacion
Notificacion
Interfaz
opt
[guardar información]
1. solicitarGuardar(archivo, datosClub)
2. validarDatos(datosClub)
3. datosValidos
4. actualizarInformación(datosClub)
5. confirmación
6. guardarArchivo(archivo, datosClub)
7. archivoGuardado
8. guardarEnBD(datosClub)
9. confirmación
10. generarNotificacion(mensaje, destinatarios)
11. confirmación
12. éxito("Información guardada")
opt
[cargar información]
13. solicitarCargar(archivo)
14. verificarExistencia(archivo)
15. archivoExiste
16. leerArchivo(archivo)
17. datosClub
18. validarDatos(datosClub)
19. datosValidos
20. cargarEnSistema(datosClub)
21. confirmación
22. mostrarInformación(datosClub)
23. confirmación
24. éxito("Información cargada")
alt
[archivo no encontrado]
25. verificarExistencia(archivo)
26. archivoNoExiste
27. error("Archivo no encontrado")

6. Decisiones de Diseño y Justificaciones

6.1 Distribución de responsabilidades

Para el diseño del Club Deportivo Séneca se distribuyeron las responsabilidades entre diferentes clases, teniendo en cuenta la información que manejan y las operaciones que realizan. El sistema se organizó en tres paquetes principales: clubdeportivo, clubdeportivo.deportes y clubdeportivo.operaciones.

ClubDeportivo: Administra socios, equipos e inscripciones. Valida cupos, categorías de edad, números de camiseta y restricciones de inscripción.

Usuario, Socio y PerfilJugador: Manejan la información de acceso, los datos básicos del socio y su identificación como jugador.

Deporte, DeporteConjunto, Categoria y CategoriaEdad: Representan los deportes de conjunto y las categorías utilizadas para organizar a los jugadores.

Equipo e InscripcionEquipo: Guardan la información de los equipos y la participación de los jugadores, incluyendo posición y número de camiseta.

Partido y PartidoInterno: Representan los encuentros deportivos, validan los equipos y permiten registrar marcadores y resultados.

Competencia, InscripcionCompetencia, FaseCompetencia y GrupoCompetencia: Organizan las competencias, sus inscripciones, fases, grupos y tablas de posiciones.

EstadisticaPartido, EstadisticaFutbol, EstadisticaBaloncesto y EstadisticaVoleibol: Registran las estadísticas individuales según el deporte, como goles, asistencias, puntos, rebotes, aces y bloqueos.

Sancion: Guarda las sanciones de los jugadores y controla los partidos de suspensión pendientes.

DeporteIndividual, Tenis y Padel: Representan las disciplinas individuales y validan sus modalidades.

PartidoPractica y RankingJugador: Manejan el registro básico de partidos de práctica y la actualización de puntajes individuales.

Ejercicio, PlanEntrenamiento y Entrenamiento: Organizan los ejercicios, los planes de entrenamiento y las sesiones programadas.

ControlEntrenamientos, RegistroAsistencia y ReporteAsistencia: Registran entrenamientos y asistencias, permiten consultar la participación de los jugadores y generar resúmenes.

EvaluacionDesempeno, ControlEvaluaciones y ReporteEvaluaciones: Guardan evaluaciones de los jugadores, calculan promedios y presentan los resultados.

HabilitacionMedica y ControlHabilitacionesMedicas: Registran las habilitaciones médicas y verifican su vigencia.

Producto, Bebida y Snack: Representan los artículos de tienda y cafetería, incluyendo características como temperatura y alérgenos.

Inventario: Administra los productos, consulta existencias y actualiza las cantidades disponibles.

Venta y DetalleVenta: Guardan la información de las transacciones y los productos incluidos en cada compra.

Caja, ReglaVenta y CuentaFidelidad: Gestionan el registro de ventas, las validaciones comerciales, los impuestos, las propinas y los puntos de fidelidad.

Instalacion, ReservaInstalacion y GestorReservas: Manejan las instalaciones, las reservas, los bloqueos de horarios y la validación de conflictos. Turno, GestorTurnos y

ValidadorTurnos: Representan los horarios del personal, gestionan cambios y verifican la cobertura mínima requerida.

ReporteVentas: Calcula los valores de ventas por rubro y periodo.

PersistenciaClub y PersistenciaOperaciones: Guardan y recuperan información mediante archivos. PersistenciaOperaciones organiza los datos operativos que se almacenan.

Main y programas de prueba: Permiten ejecutar y comprobar funcionalidades del sistema mediante la consola y casos de prueba. Las clases PartidoOficial y TenisDeMesa también están presentes en el proyecto, pero todavía no tienen comportamiento desarrollado en la versión revisada.

6.2 Justificaciones de las deciciones

Separación de responsabilidades: Se decidió dividir el sistema en diferentes clases según sus funciones. ClubDeportivo se encarga de las operaciones relacionadas con socios y equipos, mientras que otras clases administran las actividades deportivas y comerciales. Esta organización evita concentrar toda la lógica en una sola clase, aunque requiere coordinar las diferentes partes del sistema.

Uso de herencia y clases abstractas: Se utilizaron clases base como Usuario, Partido, EstadisticaPartido y DeporteIndividual para compartir características y definir comportamientos comunes. Esto reduce la repetición de información y permite que las clases especializadas manejen sus propias funcionalidades.

Organización de estadísticas deportivas: Se separaron las estadísticas de fútbol, baloncesto y voleibol porque cada deporte registra información diferente. Así, cada clase

maneja únicamente los datos que le corresponden y se facilita la incorporación de nuevas estadísticas.

Separación de ventas e inventario: Se decidió manejar los productos, el inventario y las ventas mediante clases diferentes. DetalleVenta permite registrar varios artículos dentro de una misma compra, mientras que Caja coordina las validaciones y la actualización de existencias. Por su parte, ReglaVenta concentra los cálculos de impuestos y propinas, evitando repetir estas operaciones.

Validación de reglas del negocio: Las validaciones se distribuyeron entre las clases encargadas de cada proceso. Por ejemplo, ClubDeportivo comprueba las condiciones de inscripción, GestorReservas verifica conflictos de horarios y ValidadorTurnos comprueba que se mantenga el personal mínimo requerido. Esto permite rechazar operaciones que no cumplen las condiciones establecidas.

Separación de la persistencia: Se utilizaron PersistenciaClub y PersistenciaOperaciones para manejar el almacenamiento de información mediante archivos. Esta decisión permite separar la lectura y escritura de datos de las demás operaciones, facilitando su mantenimiento. Actualmente, el almacenamiento integrado se concentra en los datos del módulo de operaciones.

Como conclusión, estas decisiones buscan mantener responsabilidades claras, reducir la repetición de código y facilitar el mantenimiento del sistema. También permiten desarrollar y probar diferentes funcionalidades por separado, aunque todavía es necesario completar la integración de algunos módulos.

7. Persistencia de la información

7.1 Manejo de la persistencia

Para conservar la información del Club Deportivo Séneca se implementó un mecanismo de persistencia mediante archivos binarios. Se utilizaron las clases PersistenciaClub y PersistenciaOperaciones, separando el almacenamiento de datos de las funcionalidades del sistema. PersistenciaClub se encarga de guardar y recuperar objetos mediante ObjectOutputStream y ObjectInputStream. Por su parte, PersistenciaOperaciones organiza la información del módulo de operaciones para almacenarla y recuperarla de forma conjunta. Los archivos se guardan en la carpeta datosClub, ubicada fuera de src, utilizando la extensión .dat.

7.2 Información almacenada

Inventario: Conserva los productos registrados y sus cantidades disponibles.

Caja: Este guarda las ventas realizadas, incluyendo sus detalles, impuestos y valores correspondientes.

Turnos: Conserva los horarios y las asignaciones registradas del personal.

Instalaciones: Guarda la información de los espacios deportivos, sus características y estados.

Estos elementos se agrupan mediante la clase interna DatosOperaciones, que permite almacenarlos en el archivo operaciones.dat.

7.3 Guardado y recuperación de datos

El proceso de almacenamiento comienza cuando PersistenciaOperaciones reúne la información del inventario, la caja, los turnos y las instalaciones. Luego utiliza PersistenciaClub para escribir los datos en el archivo correspondiente. Para recuperar la información, el sistema lee el archivo y reconstruye los objetos almacenados. También se realizan validaciones del nombre del archivo y del tipo de información recuperada. Se incluyó el programa PruebaPersistenciaOperaciones, que permite comprobar el guardado y la carga de una venta y las existencias del inventario. Por el momento, este mecanismo se encuentra integrado con el módulo de operaciones. Todavía queda pendiente incorporar la persistencia de la información deportiva y de otros elementos del club, como socios, equipos, inscripciones y reservas.

8. Consistencia entre diseño e implementación

8.1 Correspondencia entre los diagramas y el código

La implementación del Club Deportivo Séneca se desarrolló tomando como referencia los diagramas UML de alto nivel y detallado. El proyecto se organizó en tres paquetes principales: clubdeportivo, clubdeportivo.deportes y clubdeportivo.operaciones, con el propósito de separar las responsabilidades y facilitar la organización del código. En la implementación se conservaron las principales relaciones de herencia y asociaciones del diseño, como las existentes entre Usuario y Socio, Deporte y DeporteConjunto, y Partido y PartidoInterno. También se desarrollaron clases para gestionar inscripciones, competencias, estadísticas, entrenamientos, ventas, inventario, reservas, turnos y persistencia. Esta organización permite relacionar las clases del código con los diferentes componentes del diseño, manteniendo una distribución de responsabilidades acorde con las funcionalidades del club.

8.2 Verificación de la implementación

Para comprobar el funcionamiento del sistema se desarrollaron diferentes programas de prueba, enfocados en las principales funcionalidades y reglas del negocio.

Main: Permite iniciar sesión como socio, consultar los equipos disponibles, realizar inscripciones y verificar restricciones como la edad, los cupos y los números de camiseta.

PruebaDeportes: Demuestra el registro de partidos internos, la validación de equipos, el registro de resultados, las estadísticas individuales y las sanciones deportivas.

PruebaIntegracion: Comprueba el registro de ventas, la actualización del inventario, la acumulación de puntos de fidelidad, las reservas de instalaciones, la validación de turnos y la vigencia de las habilitaciones médicas.

PruebaPersistenciaOperaciones: Verifica el almacenamiento y la recuperación de información relacionada con ventas e inventario mediante archivos.